Experiment-1(route finding with A*search)

Aim:

To find the optimal path from a start node to a goal node using the A* search algorithm.

Algorithm:

1. Initialize a priority queue with the start node, cost = 0, and heuristic value.
2. While the queue is not empty:
    - Pop the node with the lowest f=g+h.
    - If it is the goal, return the path and cost.
    - Otherwise, expand its neighbors and calculate their f-values.
    - Push neighbors into the priority queue if not visited.
3. If the queue empties without reaching the goal, report failure.

Python Program:

```
python
import heapq

graph = {
   'A': [('B', 1), ('C', 4)],
   'B': [('D', 2), ('E', 5)],
   'C': [('E', 1)],
   'D': [('F', 3)],
   'E': [('F', 2)],
   'F': []
}

h = {
   'A': 6,
   'B': 5,
   'C': 3,
   'D': 3,
   'E': 1,
   'F': 0
}

def astar(start, goal):
   pq = [(h[start], 0, start, [start])]
   visited = set()

   while pq:
      f, cost, node, path = heapq.heappop(pq)

      if node == goal:
```

```
            print("Best route:", path)
            print("Total cost:", cost)
            return

        if node in visited:
            continue

        visited.add(node)

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                new_cost = cost + weight
                priority = new_cost + h[neighbor]
                heapq.heappush(
                    pq,
                    (priority, new_cost, neighbor,
                     path + [neighbor])
                )

    print("Route not found")

astar('A', 'F')
```

Sample Output:

Code
Best route: ['A', 'C', 'E', 'F']
Total cost: 7